Lab 01 Python Essentials for NLP

Task 01: Strings as Text Storage

Topic: String (str) Core Python Data Structure

(a) Store the sentence "Artificial Intelligence is transforming Natural Language Processing" in a variable called sentence. Print the sentence, its data type using type(), and its total number of characters using len().

(b) Using the sentence from part (a), extract and print: the first word using slicing, the last character using negative indexing, and the word "Language" using a specific slice range.

(c) Check whether the word "Intelligence" exists in the sentence using the in operator. Then find and print its starting index position using an appropriate string method. Print True or False for the existence check.

(d) Create a multi-line string called paragraph that contains three different NLP-related sentences. Split the paragraph by newline character to get a list of sentences and print each sentence on its own line.

Task 02: Lists for Token Management

Topic: List (list) Core Python Data Structure

(a) Create a list called tokens containing the words ["deep", "learning", "helps", "machines", "understand", "text"]. Print the full list, its length, the element at index 2, and the last element using negative indexing.

(b) Perform and print the result of each of these operations on your tokens list: (i) append the word "efficiently" to the end, (ii) insert "computers" at index 3, (iii) remove the word "helps" by value.

(c) Create two token lists — tokens_A = ["NLP", "uses", "Python"] and tokens_B = ["for", "text", "analysis"]. Concatenate them into one list called full_tokens using the + operator. Then print the combined list and use slicing to print only the last three elements.

(d) Using full_tokens from part (c), sort the list alphabetically and store it as sorted_tokens. Then use the sorted() function with key=len to sort by word length and store it as length_sorted. Print both sorted lists and explain in a comment which sorting is more useful in NLP and why.

Task 03: Dictionaries for Word Mapping

Topic: Dictionary (dict) Core Python Data Structure

(a) Build a dictionary called word_freq that stores the following word–frequency pairs: "machine": 4, "learning": 7, "data": 5, "model": 3, "text": 6. Print the dictionary, retrieve the frequency of "learning", and update the frequency of "data" to 9.

(b) Add two new entries to word_freq — "neural": 8 and "network": 5. Then delete the entry "model" using the del keyword. Finally, check whether the key "text" exists in the dictionary using the in operator and print a suitable message.

(c) Create a word-to-ID dictionary called vocab = {"nlp": 0, "python": 1, "token": 2, "vector": 3, "corpus": 4}. Then build its reverse — id_to_word — where IDs are keys and words are values. Print both dictionaries.

(d) Loop through the word_freq dictionary and print each key–value pair in the format: Word: "machine" → Frequency: 4. At the end, print the total number of unique words stored in the dictionary.

Task 04: Sets for Unique Vocabulary

Topic: Set (set) Core Python Data Structure

(a) Create a list called raw = ["nlp", "ai", "nlp", "deep", "learning", "ai", "nlp", "python"]. Convert it into a set called vocab. Print both raw and vocab, and print the number of unique words. Explain in a comment why sets remove duplicates automatically.

(b) Create set_A = {"machine", "learning", "data", "model"} and set_B = {"deep", "learning", "neural", "data"}. Compute and print: (i) the union of both sets, (ii) the intersection (words in common), and (iii) the difference set_A − set_B (words only in A).

(c) Define a stopwords set: stopwords = {"is", "the", "a", "in", "of", "and", "to"}. Given the token list tokens = ["nlp", "is", "a", "field", "of", "ai", "and", "the", "future"], use set difference to remove all stopwords and print the cleaned tokens.

(d) Starting from your vocab set in part (a), add the words "transformer" and "bert" using the appropriate method. Remove "python" using the discard() method. Then use the in operator to check whether "ai" is still present, and print the final set.

Task 05: Tuples for Fixed Linguistic Data

Topic: Tuple (tuple) Core Python Data Structure

(a) Create a list called pos_tags containing these word–tag tuples: ("language", "NOUN"), ("processes", "VERB"), ("human", "ADJ"), ("text", "NOUN"), ("quickly", "ADV"). Print the entire list, then access and separately print the word and tag from the second tuple using indexing.

(b) Unpack the first tuple in pos_tags into two variables named word and tag. Print them individually. Then write a try-except block that attempts to change the tag of the first tuple to "VERB" and prints the exact error message that Python raises.

(c) Write a function called extract_nouns(pos_list) that takes a list of (word, tag) tuples and returns a new list containing only the words where the tag is "NOUN". Call this function on pos_tags and print the returned list.

(d) Create a tuple called pipeline_steps = ("load_data", "clean_text", "tokenize", "remove_stopwords", "vectorize"). Print the full pipeline. Use the index() method to find the position of "tokenize". Print the total number of steps, and demonstrate that the tuple cannot be modified by trying to delete one element inside a try-except block.

Task 06: String Cleaning Methods

Topic: String Methods lower, upper, strip, split, join, replace

(a) Given text = " MACHINE Learning is AMAZING!!! ", apply these methods one by one and print the result after each step: (i) strip() to remove leading and trailing spaces, (ii) lower() to normalise case, (iii) replace() to remove all "!" characters.

(b) Take the cleaned text from part (a). Use split() to tokenise it into a list of words and print the list. Then use upper() on each word inside a list comprehension and print the uppercase list. Finally, use join() with " | " as separator to combine the uppercase words back into a single string.

(c) Create the string sentence = "I love Java and Java is popular". Use replace() to substitute every occurrence of "Java" with "Python". Then use find() to locate the starting position of the first occurrence of "Python" in the updated sentence and print it.

(d) Write a function called normalize(text) that takes a raw string and returns a clean version by: stripping whitespace, converting to lowercase, replacing commas and full-stops with an empty string. Test the function on at least two messy input strings and print the cleaned output for each.

Task 07: String Validation Methods

Topic: String Methods find, startswith, endswith, isdigit, isalpha

(a) Given text = "NLP is the future of AI research", use startswith() to check whether the text begins with "NLP", and endswith() to check whether it ends with "research". Print a clear message for each result. Then use find() to get the position of the word "future" in the string.

(b) Create a mixed list: tokens = ["hello", "123", "world", "456abc", "python", "789", "NLP3"]. Loop through the list and for each token print whether it is purely alphabetic (isalpha()), purely numeric (isdigit()), or mixed. Label each output clearly.

(c) Given the list words = ["running", "jumped", "NLP", "ai2025", "data", "42"]. Use a loop with isalpha() and isdigit() to separate the list into three groups: pure_words, pure_numbers, and mixed_tokens. Print each group.

(d) Write a function called validate_token(token) that returns the string "WORD" if the token is purely alphabetic, "NUMBER" if it is purely numeric, and "MIXED" otherwise. Apply this function to each element in the list ["chat", "99", "gpt4", "bert", "2024", "nlp!"] using a loop and print the token alongside its label.

Task 08: Functions and Lambda for NLP Pipelines

Topic: Functions, Lambda Reusable Text Processing

(a) Write a regular function called clean(text) that takes a string, strips whitespace, converts it to lowercase, and returns the result. Then write an equivalent lambda function called clean_lambda that performs the same two operations in one line. Test both on the string " HELLO Natural Language Processing " and print their outputs.

(b) Write a function called tokenize(text) that takes a cleaned string and returns a list of individual word tokens using split(). Then write another function called remove_short(tokens, n) that takes the token list and an integer n, and returns only tokens whose length is greater than n. Chain both functions on the sentence "I love working with NLP tools in Python daily" with n = 3 and print the result.

(c) Write a function called word_lengths(tokens) that takes a list of tokens and returns a dictionary where each key is a word and each value is the length of that word. Call the function on tokens = ["language", "model", "ai", "processing", "deep"] and print the resulting dictionary.

(d) Combine the functions from parts (b) and (c) into a full pipeline function called nlp_pipeline(text, min_len) that: cleans the text, tokenises it, removes short words, and returns a length dictionary. Call this pipeline on two different sentences and print the outputs.

Task 09: map(), filter(), sorted() and Counter

Topic: map(), filter(), sorted(), Lambda, collections.Counter

(a) Given words = ["Python", "NLP", "is", "a", "powerful", "tool", "for", "text"]. Use map() with a lambda to: (i) convert every word to lowercase — print the result as a list, and (ii) compute the character length of each word — print the result as a list.

(b) Using the same words list, apply filter() with a lambda to create two filtered lists: (i) words_long — only words with more than 3 characters, and (ii) words_with_t — only words containing the letter "t" (case-insensitive). Print both lists.

(c) Sort the words list using sorted() in three different ways and print each result with a label: (i) alphabetically (default order), (ii) by word length ascending using key=len, (iii) by word length descending using reverse=True.

(d) Import Counter from collections. Given the sentence "nlp nlp python ai nlp python text ai nlp python text", split it into words and create a Counter called freq. Print the full counter, then print: (i) the 3 most common words using most_common(), (ii) the frequency of the word "python", and (iii) the total number of words and the size of the unique vocabulary.